

CloakRFQ

Security Review



18 August, 2026

Prepared by: Valves Security

Lead Auditors: Vesko210 | Merulez99

Contents

1	About Valves Security	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	4
5	Executive Summary	4
6	Findings	5
	High Findings	5
	[H-01] Compliance and risk certificates are never fetched on-ledger, so the vetting gate is cosmetic	5
	[H-02] Settlement is not atomic and nothing enforces an independent registrar, so the seller keeps both cash and the receivable	6
	Medium Findings	7
	[M-01] Seller sets scope and policy version on assessor-signed certificates, and non-consuming choices let them mint unlimited copies	7
	[M-02] Deadline Gates Compare Submitter-Shiftable Ledger Time at an Exact Boundary	8
	Low/Info Findings	9
	[L-01] Payment-Leg Cardinality Differs Between Submit and Settle, Permanently Brick-ing a Valid Quote	9
	[L-02] A Firm Private Quote Is Unilaterally Revocable by the Funder During the Seller's Mandatory Wait	10
	[L-03] One Receivable Backs Unlimited Concurrent Quotes and Losing Funders' Capital Is Stranded	10
	[L-04] Funding Is Bound to Receivable Terms Only, Not to Debtor Identity	11
	[L-05] A Receivable Can Be Created Already-Pending, Enabling Zero-Consideration Seizure	12
	[I-01] Settlement Safety Rests on Unenforced Canton Deployment Assumptions	12
	[I-02] Seller Is Over-Divulged the Funder's Entire Allocation Payload	12
	[I-03] The Settlement-Success Gate Is Vacuous on an Empty Result List and Not Bound to the Funding Leg	13

1 About Valves Security

Valves Security is a Web3 security team founded by Veselin and Valeri. With deep experience across EVM-based protocols, Canton Network and Rust systems, we help projects identify and eliminate any vulnerabilities. With a strong background and a strong team of senior contractors we can assure a top quality audit.

Book a Security Review at valvesecurity.com or message us on X Valves Security

2 Disclaimer

The Valves team makes all effort to find as many vulnerabilities in the code in the given time period. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation. We stand by our process, yet encourage follow-up audits and meaningful bug bounty incentives for continued whitehat protection.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- High - leads to a significant material loss of assets in the protocol or a group of users
- Medium - leads to a moderate material loss of assets in the protocol, moderately harms a group of users or to some disruption of the protocol's functionality
- Low - No funds at risk

Likelihood

- High - attack path is possible with reasonable assumptions. The issue is almost certain to happen.
- Medium - only a conditionally incentivized attack vector, but still relatively likely

- Low - has unlikely assumptions or requires a significant stake by the attacker

4 Protocol Summary

Private invoice-financing RFQs on Canton.

5 Executive Summary

Overview

Project	CloakRFQ
Commit Hash	-
Timeline	16/08/2026 – 18/08/2026

Scope

Id	Files in scope
1	daml/*

Issues Found

Critical Risk	0
High Risk	2
Medium Risk	2
Low/Info Risk	8

6 Findings

High Findings

[H-01] Compliance and risk certificates are never fetched on-ledger, so the vetting gate is cosmetic

Description

The protocol's core promise — only compliance- and risk-vetted receivables are financed - is unenforced. `RFQRequest` stores `complianceCertificateCid`, `riskCertificateCid`, `complianceParty`, and `riskAssessor`, and the module comment says verifiers “must compare [them] against the request fields,” but no choice ever fetches them.

Request template (`contracts/daml/CloakRFQ/RFQRequest.daml`, lines 62 to 103):

```

1  template RFQRequest
2    with
3      seller : Party
4      funder : Party
5      complianceParty : Party
6      riskAssessor : Party
7      packageId : Text
8      receivableCid : ContractId Receivable
9      packageData : RFQPackageData
10     complianceCertificateCid : ContractId ComplianceCertificate
11     riskCertificateCid : ContractId RiskCertificate
12   where
13     signatory seller
14     observer funder
15     ensure hasValidReceivableTerms packageData.receivableTerms
16
17     choice SubmitPrivateQuote : ContractId PrivateQuote
18       with
19         quoteTerms : QuoteTerms
20         fundingAllocationCid : ContractId Token.Allocation
21         controller funder
22       do
23         now <- getTime
24         ...
25         assertMsg "allocation must contain a matching sender-side payment leg"
26           (any (matchesPaymentLeg seller packageData.paymentInstrumentId quoteTerms.
27             netPurchasePrice)
28             allocationSpec.transferLegSides)
```

`SubmitPrivateQuote` and `AcceptAndSettle` validate the funding allocation, the receivable, and the settlement factory, yet never read either certificate cid. `packageData.riskTier`, which the funder prices against, is a bare seller-asserted field never compared to `RiskCertificate.certifiedRiskTier`. The broken invariant is: a settled RFQ was compliance/risk-approved for the same receivable at the tier it advertises.

The intended off-ledger mitigation is structurally impossible. The certificates are `observer seller` only (`Risk.daml`, line 50; `Compliance.daml`, line 51), so the funder is not a stakeholder and cannot fetch them, and the assessors are not stakeholders of `RFQRequest`. The only party who can see both a certificate and the request is the seller - the adversary.

Impact

- A seller can carry an ineligible, non-existent, mis-tiered, or cross-bound receivable to full settlement; the funder commits and loses real `netPurchasePrice` against an obligation whose compliance and risk were never checked, and the audit trail cannot prove any check occurred.
- A relying party (or a future on-ledger consumer) that trusts `certifiedReceivableTerms` is shown assessor-signed terms that need not match the real receivable — mispricing or mis-eligibility rooted in an unbound attestation.
- A regulator / auditor relying on `ReceivableSaleSettlement` cannot distinguish a compliance-skipped settlement from a legitimate one; the claimed “Scoped Compliance Receipt” degrades to a bare trade receipt — false audit confidence by omission.
- Compliance / risk credentials outlive the judgment they encode; revocation is race-prone, non-atomic, and — given the gate is cosmetic — ineffective.

Recommendation

In `SubmitPrivateQuote`, and again in `AcceptAndSettle`, fetch both certificates and `assertMsg` that each is signed by the stored `complianceParty` / `riskAssessor`, references this `receivableCid` and `packageId`, and that `certifiedReceivableTerms == packageData.receivableTerms` and `certifiedRiskTier == packageData.riskTier`. Add `funder` as an observer of both certificates (or disclose them) so off-ledger verification is at least possible, and record the validated certificate references on `ReceivableSaleSettlement` (see M-04). Guard with a `submitMustFail` on `SubmitPrivateQuote` when the advertised tier differs from the certified tier.

[H-02] Settlement is not atomic and nothing enforces an independent registrar, so the seller keeps both cash and the receivable

Description

Delivery-versus-payment is not atomic, and its safety rests on an unenforced invariant. `AcceptAndSettle` settles the funder’s payment irreversibly and then only proposes the receivable transfer.

Settlement code (`contracts/daml/CloakRFQ/RFQRequest.daml`, lines 165 to 178):

```

1  settlementBatchResult <- exercise settlementFactoryCid Token.SettlementFactory_SettleBatch
   with
2    settlement = allocationView.settlement
3    transferLegs = paymentTransferLegs
4    allocations = (Token.FinalizedAllocation with
5      allocationCid = fundingAllocationCid
6      extraTransferLegSides = []
7      nextIterationFunding = None) :: extraSettlementAllocations
8    actors = allocationSettleActors allocationView
9    extraArgs = emptyExtraArgs
10  assertMsg "allocation settlement did not settle"
11    (allAllocationResultsSettled settlementBatchResult.allocationSettleResults)
12
13  receivableTransferCid <- exercise receivableCid Transfer with
14    proposedNewOwner = funder

```

The cash leg settles first (external `SettleBatch`, gated by `allAllocationResultsSettled`); the receivable is then delivered only as a pending proposal. `ReceivableSaleSettlement` is stamped `settledAt = now` while title has not moved — the funder must later exercise `AcceptTransfer` to obtain it.

Asset template (`contracts/daml/CloakRFQ/Receivable.daml`, lines 9 to 30):

```

1  template Receivable
2    with

```

```

3   registrar : Party
4   owner : Party
5   newOwner : Party
6   ...
7   where
8     signatory registrar, owner
9     observer newOwner
10    ensure isValidReceivableTerms terms
11
12    choice Transfer : ContractId Receivable
13      with
14        proposedNewOwner : Party
15        controller owner
16      do
17        assertMsg "transfer already pending" (owner == newOwner)
18        create this with newOwner = proposedNewOwner

```

`Transfer` sets `newOwner = funder` but leaves `owner = seller`, who stays a signatory. Nothing enforces `registrar /= seller` (line 20 validates only terms). Every fixture ships `registrar = seller`, so the pending receivable's entire signatory set is `{seller}` and the seller alone can `Archive` it after being paid. There is no reclaim choice anywhere on `Receivable` or `ReceivableSaleSettlement`.

Impact The winning funder loses both the `netPurchasePrice` and the receivable, unrecoverably. Under the shipped self-registrar configuration this is seller-unilateral and trivially reachable; if `registrar` is instead a genuinely independent custodian, the archive requires registrar+seller collusion racing the funder's `AcceptTransfer` — a Medium. The design comment intends an independent registrar, the code does not enforce it, so the worst realistic and tested state is High.

Recommendation

Enforce an independent registrar (`ensure registrar /= owner`, and treat the registrar as a genuinely separate custodian party). Make delivery atomic with payment — restructure so the funder co-authorizes `AcceptAndSettle` and the receivable moves to the funder in the same transaction — or gate `ReceivableSaleSettlement` creation on a completed `AcceptTransfer`. Add a funder-controlled reclaim path for the post-payment / pre-delivery window.

Medium Findings

[M-01] Seller sets scope and policy version on assessor-signed certificates, and nonconsuming choices let them mint unlimited copies

Description

`CreateRiskCertificate` and `CreateComplianceCertificate` are `nonconsuming` choices controlled by the seller, yet the certificates they produce are signed by the risk assessor or compliance party.

`CreateRiskCertificate` (`contracts/daml/CloakRFQ/Risk.daml`, lines 20 to 34):

```

1   nonconsuming choice CreateRiskCertificate : ContractId RiskCertificate
2   with
3     riskPolicyVersion : Text
4     certificationScope : Text
5     controller seller
6   do
7     create RiskCertificate with
8       riskAssessor
9       seller

```

```

10     packageId
11     receivableCid
12     certifiedReceivableTerms = riskDisclosure.receivableTerms
13     certifiedRiskTier = riskResult.riskTier
14     riskPolicyVersion
15     certificationScope

```

The `certifiedReceivableTerms` and `certifiedRiskTier` are copied faithfully from the assessor-signed attestation, so the seller cannot forge those. But `riskPolicyVersion` / `policyVersion` and `certificationScope` come verbatim from the seller's choice arguments. The seller thus binds arbitrary strings into a credential bearing the assessor's signature, and because the choice is `nonconsuming`, can mint unlimited divergent certificates from one attestation. The forgery surface is exactly those two free-text fields; the signatory attested to a disclosure and a result, not to that scope or version.

Impact

The risk / compliance assessor is placed on record, under their own signature, endorsing a policy version and certification scope they never authored — a credential-integrity break. Harm is off-ledger / reputational today because the certificates are never consumed on-ledger (see H-01); it becomes load-bearing once H-01 is fixed.

Recommendation

Move `policyVersion` / `certificationScope` into the assessor-signed attestation fields and copy them into the certificate from there, as `certifiedReceivableTerms` already is. Make the mint choice `consuming` (or key it) so one attestation yields one certificate, or require the assessor to co-authorize the certificate content. Guard with a `submitMustFail` on a second `CreateRiskCertificate` from the same attestation, and assert the certificate scope and version equal the assessor-set values.

[M-02] Deadline Gates Compare Submitter-Shiftable Ledger Time at an Exact Boundary

Description

The quote lifecycle is gated on `now <- getTime`, which returns ledger time — the submitting party can shift it within `ledgerTimeRecordTimeTolerance`; it is not wall-clock time. Every gate is a bare `<=` / `>` comparison placed at the exact boundary.

`SubmitPrivateQuote`, controlled by the funder (`contracts/dam1/CloakRFQ/RFQRequest.dam1`, lines 84 to 87):

```

1     now <- getTime
2     assertMsg "quote terms must be valid for this request" (hasValidQuoteForRequest
   packageData quoteTerms)
3     assertMsg "quote response deadline passed" (now <= packageData.responseDeadline)
4     assertMsg "quote already expired" (now <= quoteTerms.quoteExpiresAt)

```

`AcceptAndSettle`, controlled by the seller (`contracts/dam1/CloakRFQ/RFQRequest.dam1`, lines 135 to 137):

```

1     now <- getTime
2     assertMsg "RFQ response deadline has not passed" (now > packageData.
   responseDeadline)
3     assertMsg "quote expired" (now <= quoteTerms.quoteExpiresAt)

```

The party who benefits at a boundary can pick a favorable `now`. The `<=` admits equality, so a quote can settle at the exact `quoteExpiresAt`; and within the tolerance the seller can settle beyond the honest wall clock, on a quote the funder believes expired.

Impact

A funder can be bound to a settlement at the instant it believed the quote expired, and up to the skew tolerance past true expiry; competing funders can be nudged out of the response window. Magnitude is bounded by the operator-configured `ledgerTimeRecordTimeTolerance` (default order of 60 seconds).

Recommendation

Use a grace band rather than an exact boundary: require `now + tolerance <= quoteExpiresAt` to settle and `now >= responseDeadline + tolerance` to gate the response window; confirm and pin `ledgerTimeRecordTimeTolerance` as tightly as the use case allows.

Low/Info Findings

[L-01] Payment-Leg Cardinality Differs Between Submit and Settle, Permanently Bricking a Valid Quote

Description

The two legs of the same DvP gate the payment leg with different quantifiers over the identical predicate.

`SubmitPrivateQuote` accepts the allocation if any (≥ 1) sender-side leg matches (`contracts/daml/CloakRFQ/RFQRequest.daml`, lines 101 to 103):

```
1      assertMsg "allocation must contain a matching sender-side payment leg"
2          (any (matchesPaymentLeg seller packageData.paymentInstrumentId quoteTerms.
3              netPurchasePrice)
4              allocationSpec.transferLegSides)
```

`AcceptAndSettle` requires exactly one (`contracts/daml/CloakRFQ/RFQRequest.daml`, lines 161 to 163):

```
1      let paymentTransferLegs = matchingPaymentTransferLegs allocationSpec.authorizer
2          seller packageData.paymentInstrumentId quoteTerms.netPurchasePrice
3          allocationSpec.transferLegSides
4      assertMsg "allocation must contain a matching sender-side payment leg"
5      (length paymentTransferLegs == 1)
```

Because the funder authors the allocation and it is re-fetched unchanged at settle, an allocation carrying two identical matching legs passes quote submission but makes settlement abort forever.

Impact

A funder can present a quote that passes every submit-time check yet is structurally impossible to settle — and the seller only discovers this after being forced to wait past `responseDeadline`, by which time competing quotes may have expired. This is zero-cost griefing / DoS of the settle path with no fund loss and no permanent lock — the funder reclaims the allocation at its external `settlementDeadline`.

Recommendation

Make both gates identical — enforce exactly-one matching leg at `SubmitPrivateQuote` (`length (matchingPaymentTransferLegs ...) == 1`), or accept ≥ 1 and deterministically select a single leg at settle. Requiring a single-leg allocation at submit.

[L-02] A Firm Private Quote Is Unilaterally Revocable by the Funder During the Seller's Mandatory Wait

Description

`PrivateQuote` has a single signatory, so the funder alone can archive it via the implicit `Archive` choice, while the seller is blocked from acting until the response deadline passes.

`PrivateQuote` template and the settle-time gate (`contracts/daml/CloakRFQ/RFQRequest.daml`, lines 114 to 136):

```

1  template PrivateQuote
2    with
3      seller : Party
4      funder : Party
5      ...
6    where
7      signatory funder
8      observer seller
9      ensure hasValidQuoteTerms quoteTerms
10
11     choice AcceptAndSettle : ContractId ReceivableSaleSettlement
12       with
13         ...
14         controller seller
15       do
16         now <- getTime
17         assertMsg "RFQ response deadline has not passed" (now > packageData.
           responseDeadline)

```

Throughout the window `[submit, responseDeadline]` the seller cannot act while the funder can freely revoke the “binding” quote. Symmetrically, the seller can strand the funder’s committed allocation by never settling — there is no timeout-reclaim.

Impact

A quote / auction-integrity break plus denial of settlement: a funder can win the seller’s selection and then revoke before the seller is permitted to accept; a seller can strand a funder’s committed capital until expiry. There is no fund loss and the position is recoverable on both sides (the allocation reclaims at its `settlementDeadline`), so severity is Low.

Recommendation

Make `PrivateQuote` signatory `funder`, `seller` (authorized at submit time, since `RFQRequest`’s signatory is the seller) so neither side can unilaterally revoke, or add an explicit bilaterally-controlled withdraw with a time-bounded reclaim. Guard with a `submitMustFail funder (Archive)` on the quote before the deadline.

[L-03] One Receivable Backs Unlimited Concurrent Quotes and Losing Funders’ Capital Is Stranded

Description

No ledger-level exclusivity binds a receivable to a single active RFQ. `RFQRequest` and `PrivateQuote` reference a `receivableCid` but nothing prevents a seller from creating N per-funder requests all pointing at the same receivable, each requiring a committed, locked allocation.

`SubmitPrivateQuote` commitment gate (`contracts/daml/CloakRFQ/RFQRequest.daml`, lines 89 to 94):

```

1      allocation <- fetch @Token.Allocation fundingAllocationCid
2      let allocationView = view @Token.Allocation allocation
3      let allocationSpec = allocationView.allocation
4      assertMsg "allocation must be committed" allocationSpec.committed
5      assertMsg "allocation deadline must cover quote expiry"
6          (allocationDeadlineCoversQuote allocationSpec.settlementDeadline quoteTerms.
           quoteExpiresAt)

```

The receivable can be sold only once — settling one archives it via the consuming `Transfer` fetch path — so the other N-1 funders' `AcceptAndSettle` calls abort at the receivable fetch. With no funder-side cancel / withdraw, those allocations stay committed until their external `settlementDeadline`.

Impact

The N-1 losing-quote funders lose the *use* of committed capital for the quote window. This is a bounded, recoverable lockup, not theft: the committed allocation is not permanently locked — after its `settlementDeadline` it can no longer be settled and the authorizer can withdraw to release the funds.

Recommendation

Bind exclusivity to the receivable at quote time (a one-active-quote-per-receivable lock or contract key), and add a funder-controlled withdraw that releases the committed allocation once the receivable settles elsewhere or the quote expires.

[L-04] Funding Is Bound to Receivable Terms Only, Not to Debtor Identity

Description

The funder prices and commits against `packageData`, which conveys only receivable terms plus a seller-asserted risk tier — never the debtor identity or invoice id.

`RFQPackageData` (`lib/daml/CloakRFQ/Lib/Package.daml`, lines 7 to 14):

```

1  data RFQPackageData = RFQPackageData
2    with
3      receivableTerms : ReceivableTerms -- Receivable terms disclosed as part of the
         package.
4      riskTier : RiskTier                -- Risk tier certified by the Risk Assessor for
         this package.
5      responseDeadline : Time            -- Shared quote response deadline for this RFQ
         event.
6      paymentInstrumentAdmin : Party     -- CIP-56 token registry/admin expected for quote
         funding.
7      paymentInstrumentId : Text         -- CIP-56 instrument id expected for quote funding,
         e.g. USD.

```

The settle-time identity check is `receivable.terms == packageData.receivableTerms` (`RFQRequest.daml`, line 143), so two receivables with identical terms but entirely different debtors are indistinguishable. This is a distinct root cause from H-01: a data-model gap, not a missing certificate fetch.

Impact

The funder seeing only a risk tier (not the raw debtor) is the intended attestation-first design. The residual worth keeping is that funding is not bound to obligor identity: the funder commits firm capital without an on-ledger obligor handle, so debtor **concentration cannot be managed** — a funder cannot detect or cap repeated exposure to the same underlying obligor across deals.

Recommendation

Bind an `invoiceId` / debtor commitment (an opaque obligor identifier is sufficient) into `packageData` and re-check it at settlement, so a funder can manage concentration without seeing the raw debtor. Guard with a `submitMustFail` when the receivable's debtor commitment differs from the package's.

[L-05] A Receivable Can Be Created Already-Pending, Enabling Zero-Consideration Seizure

Description

`Receivable`'s `ensure` validates only terms, so a receivable can be created “born pending” with `owner = seller`, `newOwner = X` where `X != seller`.

`ensure` and `AcceptTransfer` (`contracts/daml/CloakRFQ/Receivable.daml`, lines 20 and 32 to 36):

```

1   ensure hasValidReceivableTerms terms
2   ...
3   choice AcceptTransfer : ContractId Receivable
4     controller newOwner
5     do
6       assertMsg "no pending transfer" (owner /= newOwner)
7       create this with owner = newOwner

```

`AcceptTransfer` (controller `newOwner`) asserts only `owner /= newOwner`, so a born-pending `newOwner` can immediately accept and become owner for zero consideration. Self-inflicted: only `registrar` + `seller` can author the born-pending contract, so it is a misconfiguration footgun rather than an attacker-forced path.

Impact

A misconfigured or careless issuance hands ownership to an arbitrary party with no payment coupling.

Recommendation

Add `ensure hasValidReceivableTerms terms && owner == newOwner` at creation, or drop `newOwner` from creation and only set it via `Transfer`. Guard with a `createMustFail` on a born-pending receivable.

[I-01] Settlement Safety Rests on Unenforced Canton Deployment Assumptions

Description

The atomic DvP couples an external Splice allocation with the CloakRFQ `Receivable` in one transaction (`contracts/daml/CloakRFQ/RFQRequest.daml`, lines 165 to 178); this is atomic only if both live on the same synchronizer with all settlement signatories co-hosted — an assumption that is unenforced and undocumented. The sealed-bid RFQ is order- and liveness-sensitive, so a single-operator or malicious sequencer / mediator can censor, delay, or reorder quotes and settlements into timeout. And internal-party key custody plus the registrar's co-signatory role (`Receivable.daml`, lines 17 to 18) place operator and registrar honesty inside the settled-custody trust base.

[I-02] Seller Is Over-Divulged the Funder's Entire Allocation Payload

Description

`AcceptAndSettle` (controller seller) fetches the funder's `Token.Allocation`, of which the seller is not a stakeholder.

Allocation fetch at settle (`contracts/daml/CloakRFQ/RFQRequest.daml`, lines 145 to 147):

```
1 allocation <- fetch @Token.Allocation fundingAllocationCid
2 let allocationView = view @Token.Allocation allocation
3 let allocationSpec = allocationView.allocation
```

DAML divulgence is all-or-nothing, so the seller witnesses the complete allocation payload — authorizer, admin, every `transferLegSide`, `settlementDeadline`, executors — not merely the single matched payment leg the logic needs.

This is standard Splice / DAML all-or-nothing disclosure, and the seller is the payment receiver the token standard intends to see the settlement — so it is Informational. It only exposes *unrelated* data if a committed allocation could carry legs beyond this trade, which it does not here (the allocation is authored for this settlement).

[I-03] The Settlement-Success Gate Is Vacuous on an Empty Result List and Not Bound to the Funding Leg

Description

The sole check tying payment success to the receivable transfer is `allAllocationResultsSettled`, which is vacuously `True` on an empty list.

Helper and its use (`contracts/daml/CloakRFQ/RFQRequest.daml`, lines 26 to 34, and 174 to 175):

```
1 isAllocationSettled : Token.AllocationResult -> Bool
2 isAllocationSettled result =
3   case result.output of
4     Token.AllocationResult_Settled _ -> True
5     _ -> False
6
7 allAllocationResultsSettled : [Token.AllocationResult] -> Bool
8 allAllocationResultsSettled results =
9   all isAllocationSettled results
```

```
1 assertMsg "allocation settlement did not settle"
2   (allAllocationResultsSettled settlementBatchResult.allocationSettleResults)
```

There is no assertion that a result for `fundingAllocationCid` is present and `Settled`, nor that result count matches input count.

Defense-in-depth only. A conforming `SettleBatch` returns one result per input allocation, and the funding leg is always the first input, so the vacuous empty case requires a registry that does not conform to the spec - and the in-repo mock returns a `Settled` result per allocation regardless.